

Computer Networks — Midterm Practice Papers

Three complete papers in the real format, followed by full marking schemes. Papers get harder: A is recall, B is recall plus small calculations, C is apply-and-trace.

FORMAT (MIRRORS THE REAL PAPER)

- There are **9 questions, 5 marks each**. Attempt **ALL** questions.
- Books and sheets are **ALLOWED**.
- Calculators, computers and communication devices are **NOT ALLOWED**.
- Answer in **ENGLISH** in the space provided **ONLY**.

HOW TO SIT THESE

Time yourself: 45 minutes per paper — five minutes a question, which is roughly what the real paper allows. Write in the boxes; if an answer does not fit, that is the signal you are writing prose instead of scoreable points.

Mark yourself honestly against the scheme afterwards. A point only scores if a marker could **see** it — so number your points rather than writing a paragraph. Every scheme below shows exactly where its five marks sit.

Paper A

Recall level · 9 questions · 45 marks · 45 minutes

A1. List the seven layers of the OSI reference model, in order from layer 7 down to layer 1. [5 marks]
Then state the main responsibility of the **transport layer** and of the **network layer**.

A2. (a) Give the definition of a **protocol**. (b) Explain what **encapsulation** means in a layered [5 marks]
architecture.

A3. (a) What is a **socket**? (b) What three things define a socket? (c) State the size of a port [5 marks]
number and the range reserved for well-known ports.

A4. Explain the difference between **HTTP/1.0** and **HTTP/1.1** in terms of connections. Your [5 marks]
answer must mention persistent connections and pipelining.

A5. (a) Which transport protocol and port number does **DNS** use? (b) Give **three** reasons DNS **[5 marks]** is implemented as a distributed database. (c) What is the difference between a **root** name server and an **authoritative** name server?

A6. Draw the **UDP header** and label every field with its size. State which fields are **optional**, **[5 marks]** and explain exactly what the **Length** field counts.

A7. Describe the TCP **three-way handshake**. Name the segment sent at each step, and the **[5 marks]** state each side is in.

A8. Compare **Stop-and-Wait**, **Go-back-N** and **Selective Repeat**. For each, state what the **[5 marks]** sender retransmits when a timeout occurs.

A9. Write out an **SMTP session** that sends one message. Include the commands and the numeric reply codes. **[5 marks]**

Paper B

Recall + short calculation · 9 questions · 45 marks · 45 minutes

B1. Show the mapping between the seven OSI layers and the TCP/IP model. State the **PDU name** used at the transport, network and data link layers. **[5 marks]**

B2. Answer briefly: (a) Who invented the World Wide Web, where, and when did the first web site go online? (b) What characterises **Web 2.0**? (c) What is the **digital divide**? (d) When was the last IPv4 block allocated? **[5 marks]**

B3. Compare **POP3** and **IMAP**. Give the port each uses and three functional differences. **[5 marks]**

B4. (a) How many connections does **FTP** use and on which ports? (b) What does the control connection carry, and what does the data connection carry? (c) In the command `PORT 10,0,0,1,48,246`, which IP address and port number is the client asking the server to connect to? **[5 marks]**

B5. A web page consists of a base HTML file and **3 referenced images** (4 objects in total). **[5 marks]**
Ignoring transmission time, calculate the total time in **RTTs** for (a) HTTP/1.0 non-persistent, fetched serially, (b) HTTP/1.1 persistent without pipelining, (c) HTTP/1.1 persistent with pipelining. **Show your working.**

B6. Explain the meaning of these TCP header fields, giving the size of each: **Sequence number, Acknowledgement number, Header length, Window.** **[5 marks]**

B7. Name **five** TCP flags and explain what each one does. **[5 marks]**

B8. Explain the difference between **flow control** and **congestion control** in TCP. **[5 marks]**

B9. What is **fast retransmission** and how does it differ from timeout-based retransmission? **[5 marks]**
What does **SACK** add?

Paper C

Apply and trace · 9 questions · 45 marks · 45 minutes

C1. Why is a **layered architecture** used? Explain what an **n-PDU** is, and what it means to say that the services defined in each layer are **independent**. [5 marks]

C2. Describe **BitTorrent**. Which model does it use, what are its three basic components, and what is a **seeder**? [5 marks]

C3. Why is **MIME** needed? What does **base64** do, and what does it cost? A **900-byte** binary attachment is base64 encoded — approximately what size does it become? [5 marks]

C4. A TCP sender transmits three segments: `SEQ=1` carrying bytes 1-100, `SEQ=101` carrying bytes 101-125, and `SEQ=126` carrying bytes 126-250. **The second segment is lost.** [5 marks]

(a) What acknowledgement does the receiver send after the first segment arrives? (b) What does it send after the third segment arrives, and why? (c) On timeout, what does a **Go-back-N** sender retransmit?

C5. Describe how a TCP connection is **closed**. Name the segments exchanged and the states each side passes through. **[5 marks]**

C6. A UDP datagram carries **40 bytes** of application data. (a) What value is in the **Length** field? (b) List the four UDP header fields. (c) Give two reasons UDP is described as **unreliable**. **[5 marks]**

C7. Explain the difference between **iterative** and **recursive** DNS resolution. Include the role of the **local name server**. **[5 marks]**

C8. Why does TCP need **congestion control**? Explain **slow start** and **AIMD**, and say why simply dividing the total bandwidth by the number of hosts is not used. **[5 marks]**

C9. Define each of the following: **host / end system, communication link, router, packet.** [5 marks]
Then explain what is meant by "the Internet is a **network of networks**".

Marking Scheme — Paper A

One mark per numbered point. Award the mark if the idea is present, even if the wording differs — but the point must be visible as a separate statement.

A1 • OSI layers and two responsibilities

1. Layers 7–5 named in order: Application, Presentation, Session.
2. Layers 4–3 named in order: Transport, Network.
3. Layers 2–1 named in order: Data Link, Physical.
4. Transport = transparent **end-to-end** (process-to-process) data transfer between end systems; the application need not care about network, topology, routing or media.
5. Network = **route packets** from one host to another, plus **addressing**.

Total 5 • Deck 01, slides 57–63

A2 • Protocol and encapsulation

1. A protocol defines the **format** of messages exchanged.
2. ...and the **order** of those messages.
3. ...and the **actions taken** on transmission and/or receipt of a message or event.
4. Encapsulation: going down the stack, each layer **prepends its own header** to the unit handed down from the layer above.
5. The receiver **removes the headers layer by layer** going up; layer n communicates with its peer using an n-PDU.

Total 5 • Deck 01, slides 51–54

A3 • Sockets and ports

1. A socket is the endpoint an application binds to in order to communicate; applications do not talk directly but **through sockets**, and a socket is **bidirectional**.
2. Defined by an **IP address**.
3. ...a **transport protocol** (TCP or UDP).
4. ...and a **port number**.
5. Port = **16-bit unsigned integer, 0–65535**; well-known ports are **0–1023**.

Total 5 • Deck 02, slides 5–7

A4 • HTTP/1.0 vs HTTP/1.1

1. HTTP/1.0 uses **non-persistent** connections: a separate TCP connection per object.
2. Sequence is connect → transfer one object → disconnect, repeated per object.
3. HTTP/1.1 uses **persistent** connections: multiple objects may be transferred during the lifetime of a single connection.
4. **Non-pipelined** = wait for each response before issuing the next request; **pipelined** = issue requests back-to-back without waiting (the default in 1.1).
5. The server closes a persistent connection once it has been idle for a certain time.

Total 5 • Deck 02, slides 15–18

A5 • DNS

1. **UDP**, port **53**.
2. Reason: no single name server could contain all domain names / **scalability**.
3. Reason: **traffic distribution**.
4. Reason: **no single point of failure** (also accept: local authority and maintenance).
5. A **root** server may hold no record for the hostname, but knows the IP address of the **authoritative** server for it; the authoritative server holds the actual record — every host is registered with one.

Total 5 • Deck 02, slides 31–35

A6 • UDP header

1. **Source port** (16 bits) and **Destination port** (16 bits).
2. **Length** (16 bits) and **Checksum** (16 bits) — 8 bytes of header in total.
3. **Source port is optional**.
4. **Checksum is optional**; destination port and length are required.
5. Length is measured **in octets and includes the UDP header**, not only the data.

Total 5 • Deck 03, slide 5

A7 • Three-way handshake

1. Client sends a segment with **SYN** set.
2. Server replies with **SYN + ACK**.
3. Client replies with **ACK**.
4. States — client: CLOSED → **SYN-SENT** → ESTABLISHED; server: **LISTEN** → **SYN-RCVD** → ESTABLISHED.
5. Purpose: confirm the destination **exists and is operational** and is **ready to communicate**, and **synchronise sequence numbers** before any data is sent.

Total 5 • Deck 03, slides 21–23

A8 • The three ARQ schemes

1. **Stop-and-Wait**: send one packet, wait for its ACK before sending the next.
2. On timeout it retransmits **that single unacknowledged packet**.
3. **Go-back-N**: pipelined; on timeout it retransmits **all packets from the lost one onward**, including ones the receiver already got correctly.
4. **Selective Repeat**: pipelined; on timeout it retransmits **only the missing packet**.
5. Trade-off: Go-back-N wastes bandwidth resending good packets; Selective Repeat is more efficient but needs receiver buffering and more bookkeeping.

Total 5 • Deck 03, slides 6–14

A9 · SMTP session

1. Server greets with **220**; client sends `HELO <host>` , server answers **250**.
2. `MAIL FROM: <sender>` → **250 Ok**.
3. `RCPT TO: <recipient>` → **250 Ok**.
4. `DATA` → **354**; message body follows, terminated by a line containing only a full stop; server answers **250 Ok: queued**.
5. `QUIT` → **221 Bye**.

Total 5 · Deck 02, slide 25

Marking Scheme — Paper B

B1 • OSI ↔ TCP/IP and PDU names

1. OSI layers **7, 6, 5** (Application, Presentation, Session) all map to the single TCP/IP **Application** layer.
2. OSI layer **4** maps to **TCP / UDP**; OSI layer **3** maps to **IP**.
3. OSI layers **2 and 1** map to the TCP/IP **Data Link** and **Physical** layers.
4. Transport PDU = **segment** (TCP) or **datagram** (UDP).
5. Network PDU = **packet**; data link PDU = **frame**.

Total 5 · Deck 01, slides 57–65

B2 • Overview facts

1. **Tim Berners-Lee**, at **CERN**, around **1989–1990**.
2. First web site went online **6 August 1991**.
3. Web 2.0: users create their own pages through a **web interface** (blogs) without needing to know HTML or own server space — enabling **ad-hoc user-to-user** communication and **democratising** the web.
4. Digital divide: **the gap between those who use computers and the Internet and those who do not**; without ICT infrastructure countries fall behind.
5. The last IPv4 block was allocated on **25 November 2019**.

Total 5 · Deck 01, slides 13, 25–26, 44, 48

B3 • POP3 vs IMAP

1. POP3 uses **TCP/110**; IMAP uses **TCP/143**.
2. POP3 **downloads** mail from the mailbox to the client.
3. IMAP **does not download** — all mail remains stored on the server.
4. IMAP supports server-side operations: read, delete, **create folders**.
5. IMAP therefore suits access from several devices and is what most web-based mail services use. (Also accept: POP3 replies are only +OK / -ERR .)

Total 5 · Deck 02, slides 28–30

B4 • FTP

1. FTP uses **two** connections.
2. **TCP/21** for control and **TCP/20** for data.
3. The control connection carries **commands and replies**; the data connection carries the **file contents**.
4. IP address = **10.0.0.1** (the first four numbers).
5. Port = **48 × 256 + 246 = 12,534**.

Total 5 · Deck 02, slides 20–22

B5 • RTT calculation

1. Recognise that under non-persistent HTTP each object costs **2 RTT** — one for the TCP handshake and one for the request/response.
2. **(a) 4 objects × 2 RTT = 8 RTT.**
3. **(b) 1 (handshake) + 1 (base page) + 3 (images, one at a time) = 5 RTT.**
4. **(c) 1 (handshake) + 1 (base page) + 1 (all 3 images pipelined) = 3 RTT.**
5. State the reason the base page always costs its own RTT: it must arrive first, because it is what **names the referenced objects.**

Total 5 • Deck 02, slides 10, 15–18

B6 • TCP header fields

1. **Sequence number (32 bits)** — the byte-stream number of the **first octet** of data carried in this segment.
2. **Acknowledgement number (32 bits)** — the sequence number of the **next octet the receiver expects** to receive.
3. **Header length (4 bits)** — the length of the TCP header.
4. ...measured in **32-bit words**, not bytes.
5. **Window (16 bits)** — the number of **octets the peer is able to receive.**

Total 5 • Deck 03, slides 17–19

B7 • TCP flags

One mark for each correctly named and explained flag, to a maximum of five:

1. **URG** — urgent; out-of-band delivery.
2. **ACK** — the acknowledgement number is valid / data has been received.
3. **PSH** — push; force delivery of the data.
4. **RST** — reset the connection.
5. **SYN** — synchronise sequence numbers. **FIN** — no more data. (Also accept ECE, CWR, NS for ECN.)

Total 5 • Deck 03, slides 18–19

B8 • Flow control vs congestion control

1. **Flow control protects the receiver:** TCP must not overflow the receiver's free buffer.
2. It uses a **sliding window**; the receiver sends its **window size with each ACK**, so the sender knows how much free buffer is available.
3. **Congestion control protects the network**, whose available bandwidth is usually far less than the traffic offered.
4. There is no explicit message: if the sender **does not receive ACKs it assumes congestion.**
5. Two basic techniques: **slow start** and **AIMD** (additive increase, multiplicative decrease).

Total 5 • Deck 03, slides 31–43

B9 • Fast retransmission and SACK

1. Normally the sender must wait for the **retransmission timer to expire** before resending.
2. Fast retransmit: **duplicate ACKs** arriving **before the timeout** indicate that a segment is missing.
3. The sender therefore retransmits **immediately, without waiting for the timer** — recovering faster.
4. ACKs are **cumulative**: if the ACK for segment $n+1$ arrives before n times out, n can be assumed received.
5. **SACK** (selective acknowledgement) lets the receiver report **which blocks it did receive** — e.g. ACK=101, SACK=[126-250] — so the sender resends only the true gap.

Total 5 • Deck 03, slides 26–28

Marking Scheme — Paper C

C1 · Layering, n-PDU, independence

1. Building software that communicates is **very complicated**; layering **divides the problem into smaller ones**.
2. Each protocol belongs to **one layer, solving one specific problem**.
3. The layer-n protocol on one device communicates with its peer through messages called **layer-n protocol data units (n-PDUs)**.
4. Layer n–1 **defines the services provided to** layer n, and layer n **relies on** layer n–1 to deliver its n-PDU.
5. Services in each layer are **independent**: a layer's internals can change without affecting the others, as long as the service it offers stays the same.

Total 5 · Deck 01, slides 51–52

C2 · BitTorrent

1. A protocol for **file distribution** using the **peer-to-peer** model.
2. There is **no centralised server** distributing the file.
3. **Metainfo** (the .torrent file): announce URL, name, length, piece length and a SHA-1 hash per piece.
4. **Tracker** (introduces peers to one another) and **Peer**.
5. While a peer is downloading chunks it **also distributes the chunks it already has**; a **seeder** is a peer holding the complete file.

Total 5 · Deck 02, slides 38–39

C3 · MIME and base64

1. SMTP supports **only 7-bit ASCII**.
2. MIME defines mechanisms for sending **other kinds of data**: 8-bit character encodings (ISO-8859-1, TIS-620) and **binary content** — files, images, audio, programs.
3. **Base64** encodes arbitrary binary data into **7-bit printable ASCII**.
4. The cost is roughly **33% larger** than the original (every 3 bytes become 4 characters).
5. **900 × 4/3 = 1200 bytes** (accept "about 1200 B" or "∼33% more").

Total 5 · Deck 02, slide 27

C4 · Sequence and acknowledgement trace

1. (a) The receiver sends **ACK = 101**.
2. Because the acknowledgement number is the **next octet expected**, and bytes 1–100 have been received.
3. (b) It sends **ACK = 101 again** — a **duplicate ACK**.
4. Because ACKs are **cumulative**: the gap at byte 101 stops the acknowledgement advancing, even though bytes 126–250 arrived correctly.
5. (c) A Go-back-N sender retransmits **everything from 101 onward** — both DATA[101–125] and DATA[126–250] .

Total 5 · Deck 03, slides 11, 24–25

C5 • Connection termination

1. The closing side sends **FIN** and enters **FIN-WAIT1**.
2. The peer replies **ACK**; the peer enters **CLOSE-WAIT** and the closer moves to **FIN-WAIT2**.
3. The peer then sends its own **FIN** and enters **LAST-ACK**.
4. The closer replies **ACK** and enters **TIMED-WAIT**.
5. After a timeout both sides reach **CLOSED**. Four segments are needed because the connection is **full duplex** — each direction must be shut down separately.

Total 5 • Deck 03, slides 29–30

C6 • UDP length and unreliability

1. (a) **Length = 48**.
2. Because the field counts **octets including the 8-byte UDP header** (40 + 8).
3. (b) **Source port, Destination port, Length, Checksum**.
4. (c) There are **no acknowledgements and no retransmission** — nothing in UDP detects or repairs loss.
5. There is also **no connection setup, no ordering and no flow or congestion control**: UDP simply adds a header and passes the datagram to IP.

Total 5 • Deck 03, slides 3–5

C7 • Iterative vs recursive DNS

1. The client sends its query to a **local name server**, which is close to the client and resolves names on its behalf.
2. **Iterative**: the queried server does not resolve the name — it returns a **referral**, i.e. the address of another server to ask.
3. The local name server then **follows each referral itself**, working from a **root** server down to the **authoritative** server.
4. **Recursive**: the queried server takes responsibility and **chases the answer on the requester's behalf**, querying other servers itself.
5. ...and returns only the **final answer**. In practice the client→local step is recursive while local→root/authoritative steps are iterative.

Total 5 • Deck 02, slides 34–37

C8 • Congestion control

1. Offered traffic is usually **much greater than the available bandwidth**.
2. Without control, hosts transmit, time out and retransmit repeatedly until **eventually nobody can transmit successfully** — so every host must sense the network state.
3. **Slow start**: begin cautiously with a small window and increase it, moving into **congestion avoidance** as the limit is approached.
4. **AIMD**: **additive increase** while no loss is detected, **multiplicative decrease** when loss indicates congestion.
5. Simply dividing total bandwidth by the number of hosts is **not efficient** — hosts are idle at different times, so a fixed share is wasted; TCP instead senses the bandwidth actually available and tries to use it.

Total 5 • Deck 03, slides 41–43

C9 • Basic terminology

1. **Host / end system**: a device that connects to the network — computers, PDAs, TVs, mobile phones, cars, even ovens and photo frames.
2. **Communication link**: what connects end systems — copper wire, fibre optics, radio spectrum.
3. **Router**: an intermediate switching device; end systems are usually connected not directly but logically **through routers**.
4. **Packet**: the chunk of information end systems transmit across the network.
5. **"Network of networks"**: the Internet interconnects sets of networks worldwide so that computers in those networks can exchange information; all end systems, routers and other components run **protocols** that control how packets are handled.

Total 5 • Deck 01, slides 9, 55–56

AFTER MARKING

Score below 35/45 on a paper? The gap is almost always **missing points, not wrong points** — you knew the material but wrote three statements where the scheme wanted five. Re-attempt the same question and force yourself to write five numbered lines.